EEA Winter 2025-26: DNN Compression & CV - Mid Eval Report

Group 13: Tharun D, Ahmed Adnan, Alok Kumar, Snehasish Haldar, Shreyansh Singh

Contents

1 Image Basics and Frequency Domain Processing **4**
1.1 Image Representation . . . 4
1.2 Grayscale Conversion . . . 4
1.2.1 Average Method . . . 4
1.2.2 Weighted Method . . . 4
1.3 The Frequency Domain . . . 4
1.3.1 DFT vs. FFT . . . 4
1.4 Magnitude and Phase . . . 5
1.5 Filtering in Frequency Domain . . . 5

2 Image Enhancement, Histograms, and Colour Spaces **6**
2.1 Image Representation and Histograms . . . 6
2.1.1 Histograms . . . 6
2.2 Colour Spaces . . . 6
2.2.1 HSV (Hue, Saturation, Value) . . . 6
2.2.2 HSL (Hue, Saturation, Lightness) . . . 6
2.3 Basic Image Operations . . . 6
2.3.1 Brightness . . . 6
2.3.2 Contrast . . . 6
2.4 Advanced Image Operations . . . 7
2.4.1 Gamma Correction . . . 7
2.4.2 White Balance . . . 7
2.4.3 Vibrance vs. Saturation . . . 7

3 Convolution, Gradients and Edge Detection **8**
3.1 The Convolution Operation . . . 8
3.2 Smoothing Filters (Blur) . . . 8
3.2.1 Average Blur . . . 8
3.2.2 Gaussian Blur . . . 8
3.3 Gradients and Edge Detection . . . 8
3.3.1 Sobel Operator . . . 8
3.3.2 Canny Edge Detector . . . 9
3.4 Sharpening . . . 9
3.4.1 Laplacian Sharpening . . . 9
3.4.2 Unsharp Masking . . . 9

4 Advanced Feature Detection (Corners and Hough) **10**
4.1 Corner Detection . . . 10
4.1.1 Harris Corner Detector . . . 10
4.2 Hough Transform . . . 10
4.2.1 Line Detection . . . 10
4.3 Generalized Hough Transform (GHT) . . . 10
4.3.1 Building the R-Table (Training Phase) . . . 11
4.3.2 Detection Algorithm (Testing Phase) . . . 11
4.3.3 Handling Scale and Rotation . . . 11

5 Feature Engineering for Computer Vision **12**
5.1 The Need for Feature Extraction . . . 12
5.2 Statistical Colour Features (6 Dimensions) . . . 12
5.2.1 Hue (Dominant Colour) . . . 12

5.2.2 Saturation (Purity) . . . 12
5.2.3 Value (Brightness) . . . 12
5.3 Geometric Shape Features (Hu Moments) . . . 13
5.4 Texture Analysis: Local Binary Patterns (LBP) . . . 13
5.4.1 The Algorithm . . . 13

6 Fundamentals of Machine Learning **14**
6.1 Linear Regression (Prediction) . . . 14
6.2 Logistic Regression (Classification) . . . 14
6.3 K-Means Clustering (Unsupervised) . . . 14

7 Deep Learning and Neural Networks **16**
7.1 Multilayer Perceptron (MLP) . . . 16
7.2 Backpropagation . . . 16
7.3 Convolutional Neural Networks (CNNs) . . . 16
7.3.1 Convolutional Layer . . . 16
7.3.2 Pooling Layer . . . 16
7.4 Training Challenges . . . 17

8 Deep Neural Network Compression **18**
8.1 The Problem with Large Models . . . 18
8.2 Pruning . . . 18
8.3 Weight Quantization and Clustering . . . 18
8.3.1 K-Means Clustering for Weight Sharing . . . 18

9 PyTorch and Automatic Differentiation **20**
9.1 Tensors: The Fundamental Unit . . . 20
9.1.1 GPU Acceleration . . . 20
9.2 Calculus of Learning . . . 20
9.3 Autograd: Automatic Differentiation . . . 20
9.3.1 The Backward Pass . . . 21
9.3.2 Controlling Gradients . . . 21
9.4 The Training Workflow . . . 21
9.4.1 Data Preparation . . . 21
9.4.2 The Iterative Loop . . . 21

1 Image Basics and Frequency Domain Processing

1.1 Image Representation

In computer vision, an image is basically stored as a numerical matrix.

- **Matrix Structure:** A standard colour image is represented as an array of dimensions $H \times W \times 3$, where H is image height, W is image width, and 3 represents the colour channels (Red, Green, Blue).

```
img[x, y]
# returns an array [R, G, B] or [B, G, R] containing colour intensities
# of the pixel at (x, y)
```

- **Data Types:** Pixel colour intensities range from 0 to 255 (stored as `uint8`). For mathematical operations (like Fourier Transforms), images are often converted to `float32`.
- **Channel Ordering:** Different libraries use different conventions. OpenCV loads images in **BGR** format, while Matplotlib and standard RGB libraries expect **RGB**. Hence, BGR to RGB conversion is a critical step.

1.2 Grayscale Conversion

Processing colour images is computationally expensive. Also, many algorithms (e.g., edge detection, FFT) operate effectively on grayscale images. There are two main conversion methods:

1.2.1 Average Method

Takes the simple arithmetic mean of the channels.

$$I_{gray} = \frac{R + G + B}{3} \tag{1}$$

1.2.2 Weighted Method

Accounts for the human eye's varying sensitivity to different colours (we are most sensitive to Green, least to Blue).

$$I_{gray} = 0.299R + 0.587G + 0.114B \tag{2}$$

1.3 The Frequency Domain

The Fourier Transform allows us to analyse an image as a sum of weighted sinusoidal waves of various frequencies.

- **Spatial Domain Matrix:** Represents pixel intensities at locations (x, y).
- **Frequency Domain:** Represents weight of frequency components (u, v).
  - **Low Frequencies:** Correspond to smooth regions, gradients and backgrounds.
  - **High Frequencies:** Correspond to edges, noise, textures and fine details.

1.3.1 DFT vs. FFT

- **DFT (Discrete Fourier Transform) Complexity:** $O(N^2)$ (Too slow for large images).
- **FFT (Fast Fourier Transform):** An optimized algorithm with complexity $O(N \log N)$, making real-time processing feasible.

1.4 Magnitude and Phase

The output of an FFT is a complex number that can be broken down into:

1. **Magnitude:** How strong a frequency component is. To visualize this, we use a logarithmic scale because the dynamic range is very large (the DC component is massive compared to high frequencies):
$$\text{Magnitude} = 20 \cdot \log(|F_{shift}| + 1) \tag{3}$$

2. **Phase:** Where the frequency components are located. Phase carries crucial structural information (shapes and edges).

1.5 Filtering in Frequency Domain

We can filter images by multiplying a mask(a matrix with 0s and 1s) to the FFT of the image and then performing an Inverse FFT.

- **Low Pass Filter (LPF):**
  - **Mask:** A circle of 1s at the center (low frequencies), 0s elsewhere.
  - **Effect:** Removes high frequencies (edges/noise), resulting in a **blurred** image.
- **High Pass Filter (HPF):**
  - **Mask:** Inverse of LPF (0 at center, 1 elsewhere).
  - **Effect:** Removes low frequencies (smooth gradients), resulting in **background removal**.

2 Image Enhancement, Histograms, and Colour Spaces

2.1 Image Representation and Histograms

2.1.1 Histograms

A histogram is a graphical representation of the distribution of pixel intensities in an image. It plots intensity values against the count of pixels (with) possessing that intensity.

Interpretation of Histograms:

- **Left-leaning:** Indicates a dark image (underexposed).
- **Right-leaning:** Indicates a bright image (overexposed).
- **Centered:** Suggests balanced exposure.
- **Narrow width:** Indicates low contrast (flat appearance).
- **Wide width:** Indicates high contrast (vivid difference between light and dark).

2.2 Colour Spaces

While RGB is the standard for hardware, other colour spaces model human perception better.

2.2.1 HSV (Hue, Saturation, Value)

- **Hue (H):** Represents the type of colour on the colour wheel (e.g., Red($0° or 360°$), Green($120°$), Blue($240°$)). In OpenCV, this is mapped to $0 - 180$.
- **Saturation (S):** Represents the purity or vividness of the colour.
- **Value (V):** Represents brightness or luminance.

2.2.2 HSL (Hue, Saturation, Lightness)

Similar to HSV, but defines Lightness (L) differently. Pure colours occur at $L = 0.5$. $L = 0$ is black, and $L = 1$ is white.

2.3 Basic Image Operations

2.3.1 Brightness

Brightness refers to the overall lightness or darkness. Mathematically, it is equivalent to adding a constant bias β to all pixel intensities:

$$I_{new}(x, y) = \text{clip}(I_{old}(x, y) + \beta, 0, 255) \tag{4}$$

Positive β shifts the histogram to the right; negative β shifts it to the left. Excessive shifting may cause clipping (loss of detail).

2.3.2 Contrast

Contrast is the difference in luminance between objects. It is adjusted by scaling the pixel values around a midpoint:

$$I_{new} = \text{clip}(\alpha \cdot (I_{old} - \text{mid}) + \text{mid}, 0, 255) \tag{5}$$

- $\alpha > 1$: Expands the histogram (increases contrast).
- $\alpha < 1$: Compresses the histogram (decreases contrast).

2.4 Advanced Image Operations

2.4.1 Gamma Correction

Human eyes are non-linearly sensitive to light (more sensitive to dark tones). To correct for display devices (like legacy CRTs which had $\gamma \approx 2.2$), images are gamma-encoded. The correction formula is:

$$V_{out} = V_{in}^{(1/\gamma)} \tag{6}$$

This operation stretches dark regions, making shadows more visible without whitewashing highlights.

2.4.2 White Balance

The goal of white balance is to remove colour casts so that white objects appear white under any lighting.

- **Gray World Assumption:** Assumes the average colour of a scene is neutral gray. We compute scale factors for each channel:

$$Scale_{Channel} = \frac{Avg_{Global}}{Avg_{Channel}} \tag{7}$$

- **White Patch Method:** Assumes the brightest pixel is perfect white and scales other pixels accordingly.

2.4.3 Vibrance vs. Saturation

- **Saturation:** Increases intensity of *all* colors equally. Can cause skin tones to look unnatural.
- **Vibrance:** Selectively boosts saturation only for pixels that are currently low-saturation. It protects already saturated pixels and skin tones.

$$S_{new} = S_{old} + \alpha \cdot (1 - S_{old}) \cdot S_{old} \tag{8}$$

3 Convolution, Gradients and Edge Detection

3.1 The Convolution Operation

Convolution is the fundamental operation in computer vision for filtering. It involves passing a kernel (a small matrix of weights) over an image. For a 3×3 kernel K and image I, the output pixel value is the weighted sum of the neighbourhood:

$$(I * K)(x, y) = \sum_{i=-1}^{1} \sum_{j=-1}^{1} I(x+i, y+j) \cdot K(i, j) \tag{9}$$

Kernel Properties:

- **Blurring:** Weights sum to 1 (preserves brightness).
- **Sharpening/Edges:** Weights sum to 0 or > 1 (modifies contrast).

3.2 Smoothing Filters (Blur)

3.2.1 Average Blur

Replaces a pixel with the mean of its neighbors.

$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

This reduces noise but eliminates fine detail.

3.2.2 Gaussian Blur

Weights decay according to a Gaussian distribution, giving more importance to the center pixel. It provides more natural smoothing than a box filter.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{10}$$

$$K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

3.3 Gradients and Edge Detection

An edge is defined as a rapid change in image intensity. We detect this by computing the gradient (derivative) of the image.

3.3.1 Sobel Operator

The Sobel operator approximates the derivative using two kernels, one for horizontal change (G_x) and one for vertical change (G_y):

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

From these, we calculate the Gradient Magnitude (S) and Direction (θ):

$$S = \sqrt{G_x^2 + G_y^2} \tag{11}$$

$$\theta = \tan^{-1}\left(\frac{G_y}{G_x}\right) \tag{12}$$

Regions where S is high correspond to edges.

3.3.2 Canny Edge Detector

The industry standard algorithm involving a multi-stage pipeline:

1. **Gaussian Blur:** Remove noise to prevent false detections.
2. **Sobel Gradients:** Compute intensity gradients (G_x, G_y).
3. **Laplacian:** Compute second order derivatives of intensity.
4. **Non-Maximum Suppression:** Thin the edges by keeping only the local maxima along the gradient direction.
5. **Double Thresholding:** Classify pixels as Strong, Weak, or Non-edges.
6. **Edge Tracking (Hysteresis):** Weak edges are kept only if they are connected to strong edges.

3.4 Sharpening

Sharpening works by enhancing the contrast at edges.

3.4.1 Laplacian Sharpening

The Laplacian ($\nabla^2 I$) is a second-order derivative operator that detects rapid intensity changes.

$$\text{Laplacian Kernel} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

To sharpen an image, we subtract the Laplacian from the original image:

$$I_{sharp} = I - \alpha \nabla^2 I \qquad (\alpha > 0) \tag{13}$$

3.4.2 Unsharp Masking

A method where a blurred version of the image (B) is subtracted from the original (I) to create a mask of details ($I - B$). This mask is added back to the original:

$$I_{sharp} = I + k(I - B) \tag{14}$$

where k controls the amount of sharpening.

4 Advanced Feature Detection (Corners and Hough)

4.1 Corner Detection

Corners are regions where intensity changes significantly in all directions.

- **Flat Region:** Shifting a window yields no change in intensity.
- **Edge:** Shifting perpendicular to the edge yields change; shifting along it yields none.
- **Corner:** Shifting in any direction yields a large change.

4.1.1 Harris Corner Detector

The Harris detector mathematically measures this change using a matrix M derived from image gradients (I_x, I_y):

$$M = \sum_{(x,y)\in W} \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \tag{15}$$

We compute a Corner Response score R:

$$R = \det(M) - k(\text{trace}(M))^2 \tag{16}$$

where $\det(M) = \lambda_1 \lambda_2$ and $\text{trace}(M) = \lambda_1 + \lambda_2$.

- If R is large and positive $\rightarrow$ Corner.
- If R is large and negative $\rightarrow$ Edge.
- If $|R|$ is small $\rightarrow$ Flat region.

4.2 Hough Transform

The Hough Transform is a technique to detect shapes (like lines or circles) that can be represented mathematically, even if the edges are broken or noisy.

4.2.1 Line Detection

A line in image space $y = mx + c$ can be represented in parameter space (Hough Space) as a point (m, c). However, vertical lines cause unbound m. Instead, we use Polar Coordinates:

$$\rho = x\cos\theta + y\sin\theta \tag{17}$$

- ρ: Perpendicular distance from origin.
- θ: Angle of the perpendicular.

Algorithm:

- Create a 2D accumulator array for (ρ, θ).
- For every edge pixel (x, y), calculate all possible (ρ, θ) pairs that pass through it.
- Vote in the accumulator array.
- Peaks in the accumulator represent detected lines.

4.3 Generalized Hough Transform (GHT)

Standard Hough Transforms rely on analytic equations (e.g., lines, circles). The Generalized Hough Transform (GHT) allows detection of arbitrary, non-analytic shapes by using a learnable template model stored in a look-up table called the **R-Table**.

4.3.1 Building the R-Table (Training Phase)

To detect an arbitrary shape, we first learn its geometry using a reference template.

1. **Reference Point:** Select a reference point (x_c, y_c) for the shape (e.g., the centroid).
2. **Boundary Analysis:** For every point (x, y) on the shape's boundary:
   - Compute the gradient orientation ϕ.
   - Calculate the vector $\vec{r}$ connecting the boundary point to the reference point. This vector is stored as a magnitude r and an angle α.
3. **Table Construction:** We populate the R-Table where the key is the gradient angle ϕ. Each entry contains a list of all $\vec{r}$ vectors associated with that edge orientation.

$$\text{R-Table}(\phi_i) = \{\vec{r}_{i,1}, \vec{r}_{i,2}, \dots\}$$

   where $\vec{r} = (r, \alpha)$ represents the distance and angle relative to the edge point.

4.3.2 Detection Algorithm (Testing Phase)

Once the R-Table is built, we can detect the shape in a new image:

1. **Initialize Accumulator:** Create a 2D array $A(x_c, y_c)$ initialized to zeros.
2. **Edge Detection:** Compute edges and their gradient orientations ϕ in the target image.
3. **Voting:** For every edge point (x_i, y_i) with orientation ϕ_i:
   - Look up ϕ_i in the R-Table to retrieve the list of vectors $\{(r_k, \alpha_k)\}$.
   - For each vector, calculate a candidate reference point (center):

$$x_c = x_i + r_k \cos(\alpha_k) \tag{18}$$

$$y_c = y_i + r_k \sin(\alpha_k) \tag{19}$$

   - Increment the accumulator: $A(x_c, y_c) += 1$.
4. **Find Peaks:** Local maxima in the accumulator correspond to the reference points of detected shapes.

4.3.3 Handling Scale and Rotation

If the object in the image is scaled by s or rotated by θ relative to the template, the basic 2D accumulator is insufficient. We must expand to a 4-dimensional parameter space $A(x_c, y_c, s, \theta)$.

The voting equations are modified to account for these transformations:

$$x_c = x_i + s \cdot r_k \cos(\alpha_k + \theta) \tag{20}$$

$$y_c = y_i + s \cdot r_k \sin(\alpha_k + \theta) \tag{21}$$

Limitations: Increasing dimensions (from 2D to 4D) causes exponential growth in memory usage and computational cost. If s and θ are unknown, GHT becomes impractical for complex shapes.

5 Feature Engineering for Computer Vision

Feature Engineering refers to the process of transforming raw image data into meaningful, robust numerical features that improve machine learning model performance.

5.1 The Need for Feature Extraction

Raw images are high-dimensional and sensitive to variations in lighting, rotation, and position. By extracting specific features, we provide the model with "domain knowledge" helping it distinguish classes more effectively.

- **Input:** Original Image (RGB).
- **Augmented Input:** Original Image + Processed Images (LBP/Filters) + Scalar Feature Vector.
- **Goal:** More input → More info → Better output.

5.2 Statistical Colour Features (6 Dimensions)

Colour is a primary discriminator for objects like fruits. However, raw RGB values are correlated and sensitive to lighting. We utilize the HSV colour space to extract 6 statistical features.

5.2.1 Hue (Dominant Colour)

Hue represents the identity of the colour (e.g., Red vs. Yellow).

- **Mean Hue (μ_H):** Identifies the object's class based on colour (e.g., Banana ≈ Yellow, Apple ≈ Red).
- **Std Deviation Hue (σ_H):** Measures colour uniformity.
  - Low σ_H: Uniformly coloured fruit (e.g., fresh banana).
  - High σ_H: Multi-coloured or spotted fruit (e.g., spotted apple).

5.2.2 Saturation (Purity)

Saturation measures the intensity or purity of the colour.

- **Mean Saturation (μ_S):** Indicates freshness. Fresh fruits typically have high, vivid saturation, while overripe or dull fruits have lower saturation.
- **Std Deviation Saturation (σ_S):** Indicates surface texture variation. Rough or uneven surfaces cause fluctuations in saturation readings.

5.2.3 Value (Brightness)

Value measures the light intensity.

- **Mean Value (μ_V):** separates dark fruits (berries) from bright fruits (lemons).
- **Std Deviation Value (σ_V):** Captures the spread of shadows and highlights. Smooth fruits have low σ_V, while bumpy fruits (like lychees) have high σ_V due to self-shadowing.

5.3 Geometric Shape Features (Hu Moments)

Standard pixel matching fails when an image is rotated or scaled. To solve this, we use Image Moments, which are weighted averages of image pixel intensities.

Hu Moments are a specific set of 7 numbers derived from central moments that possess the following invariance properties:

1. **Translation Invariance:** Moving the object inside the frame does not change the values.
2. **Scale Invariance:** Zooming in or out does not change the values.
3. **Rotation Invariance:** Rotating the object does not change the values.

This ensures that a vertical banana and a horizontal banana are recognized as the same object.

5.4 Texture Analysis: Local Binary Patterns (LBP)

Local Binary Patterns (LBP) is a powerful, computationally efficient texture descriptor that is robust to monotonic lighting changes. It captures the microstructure of the image (e.g., smooth skin vs. rough peel).

5.4.1 The Algorithm

For every pixel (x, y) in a grayscale image:

1. **Neighborhood:** Select a 3×3 window around the central pixel.
2. **Thresholding:** Compare the center pixel's intensity (I_c) with its 8 neighbours (I_n).

$$S(I_n) = \begin{cases} 1 & \text{if } I_n \geq I_c \\ 0 & \text{if } I_n < I_c \end{cases} \tag{22}$$

3. **Binary String:** Concatenate the 8 binary results (clockwise or counter-clockwise) to form an 8-bit binary number.
4. **Decimal Conversion:** Convert this binary number to a decimal value $(0 - 255)$.

$$LBP_{P,R} = \sum_{p=0}^{P-1} S(I_p - I_c)2^p \tag{23}$$

This value replaces the central pixel in the LBP output image.

6 Fundamentals of Machine Learning

6.1 Linear Regression (Prediction)

Goal: Model a linear relationship between input features x (n-dimensional vector) and a continuous target y.

Hypothesis:

$$\hat{y} = h_w(x) = w^T x \tag{24}$$

where w represents the learned weights.

Objective Function: We minimize the Mean Squared Error (MSE):

$$L(w) = \frac{1}{N} \sum_{n=1}^{N} (y_n - w^T x_n)^2 \tag{25}$$

Optimization:

- **Gradient Descent:** Iteratively update weights: $w_{t+1} = w_t - \eta \nabla L(w)$.
- **Normal Equation:** A closed-form analytical solution:

  $$w = (X^T X)^{-1} X^T y \tag{26}$$

  While the Normal Equation is direct, it is computationally expensive ($O(n^3)$) for large feature sets compared to Gradient Descent.

6.2 Logistic Regression (Classification)

Goal: Predict the probability that an input belongs to a specific class (binary $y \in \{0, 1\}$). **Hypothesis:** Applies the Sigmoid function $\sigma(z)$ to squash the linear output into range $[0, 1]$:

$$\hat{y} = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}} \tag{27}$$

Objective Function: We minimize Binary Cross-Entropy (Negative Log-Likelihood):

$$L(w) = -\frac{1}{N} \sum_{n=1}^{N} [y_n \log(\hat{y}_n) + (1 - y_n) \log(1 - \hat{y}_n)] \tag{28}$$

This function penalizes confident wrong predictions heavily.

Multi-Class Extension: For $C > 2$ classes, the **Softmax** function is used to produce a probability distribution:

$$p(y = i|x) = \frac{e^{w_i^T x}}{\sum_{j=1}^{C} e^{w_j^T x}} \tag{29}$$

6.3 K-Means Clustering (Unsupervised)

Goal: Partition N data points into K distinct clusters based on similarity. **Objective:** Minimize the Within-Cluster Sum of Squares (WCSS):

$$J = \sum_{k=1}^{K} \sum_{x_n \in C_k} ||x_n - \mu_k||^2 \tag{30}$$

where μ_k is the centroid of cluster k.

Lloyd's Algorithm:

1. **Assignment:** Assign every point to the nearest centroid.
2. **Update:** Recalculate centroids as the mean of assigned points.
3. Repeat until convergence.

Optimization: Use the "Elbow Method" (plotting WCSS vs K) to find the optimal number of clusters.

7 Deep Learning and Neural Networks

7.1 Multilayer Perceptron (MLP)

An MLP is a universal function approximator capable of learning complex non-linear relationships.

Neuron Model: A single neuron computes a weighted sum plus a bias, followed by a non-linear activation:

$$a = f(z) = f\left(\sum_{i=1}^{D} w_i x_i + b\right) \tag{31}$$

Common Activation Functions:

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$ (Values in $[0, 1]$).
- **Tanh:** $\tanh(z)$ (Values in $[-1, 1]$).
- **ReLU (Rectified Linear Unit):** $f(z) = \max(0, z)$. This is widely used in deep learning.

7.2 Backpropagation

Backpropagation is the algorithm used to train neural networks. It computes the gradient of the loss function L with respect to every weight by applying the **Chain Rule** backward from the output layer to the input layer.

$$\frac{\partial L}{\partial w^{(l)}} = \frac{\partial L}{\partial a^{(L)}} \cdot \dots \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w^{(l)}} \tag{32}$$

These gradients are then used to update weights via Gradient Descent.

7.3 Convolutional Neural Networks (CNNs)

CNNs are designed for grid-like data (e.g., images). They solve the scalability issues of MLPs by using Parameter Sharing and Translation Invariance.

7.3.1 Convolutional Layer

Instead of fully connected layers, CNNs use sliding filters (kernels).

$$G[i, j] = \sum_m \sum_n I[i - m, j - n] \cdot K[m, n] \tag{33}$$

This allows the network to learn local features like edges and textures regardless of their position in the image.

7.3.2 Pooling Layer

Pooling reduces the spatial dimensions of feature maps, making the model more robust to small shifts/translations and reducing computational cost.

- **Max Pooling:** Selects the maximum value in a window ($y = \max(x_i)$).
- **Average Pooling:** Computes the average value in a window.

7.4 Training Challenges

Overfitting: Occurs when the model memorizes training data but fails to generalize to unseen test data.

- **Detection:** Training loss decreases while Validation loss increases.
- **Solutions:** Regularization (L2, Dropout), more data, or simpler architectures.

8 Deep Neural Network Compression

Deep Learning models, especially CNNs and Transformers, are computationally expensive and memory-intensive. The following are techniques to compress these models for deployment on resource-constrained devices (like mobiles or embedded systems).

8.1 The Problem with Large Models

Modern Neural Networks have millions of parameters (Weights W).

- **Storage Cost:** Large files are hard to distribute.
- **Memory (RAM) Usage:** Loading huge matrices requires significant RAM, which is scarce on edge devices.
- **Computations:** High FLOPs (Floating Point Operations) lead to slow inference and high battery drain.

8.2 Pruning

Pruning involves removing unnecessary connections (weights) from the network to create a sparse model.

- **Concept:** Many weights in a trained network are very close to zero and contribute little to the output.
- **Method:** Set weights $|w| <$ threshold to zero.
- **Result:** It is possible to remove up to 80% of connections without significant loss in accuracy.
- **Sparse Storage:** Instead of storing a full matrix, we only store the non-zero values and their indices.

8.3 Weight Quantization and Clustering

Quantization reduces the precision of the weights to save space (e.g., converting 32-bit floats to 8-bit integers).

8.3.1 K-Means Clustering for Weight Sharing

Instead of storing every unique weight value, we cluster similar weights together.

1. **Cluster Weights:** Use K-Means to group all weights in a layer into k clusters (e.g., $k = 16$).
2. **Codebook:** Store only the k centroids (representative values).
3. **Index Map:** Replace the original weight matrix with a matrix of indices pointing to the codebook.

Training with Shared Weights: When updating weights during backpropagation:

1. Calculate gradients for all weights.
2. Group gradients by cluster index.
3. Sum the gradients for each cluster.

4. Update the centroid value using the summed gradient.

$$\frac{\partial L}{\partial \mu_k} = \sum_{w_i \in C_k} \frac{\partial L}{\partial w_i} \tag{34}$$

This ensures that all weights in a cluster remain identical, preserving the compression ratio.

9 PyTorch and Automatic Differentiation

PyTorch is the industry-standard framework for deep learning. Unlike older static graph frameworks (like early TensorFlow), PyTorch uses a Python-first philosophy with dynamic computation graphs, allowing for easier debugging and iteration.

9.1 Tensors: The Fundamental Unit

A Tensor is the core data structure in PyTorch. Mathematically, it is a generalization of vectors and matrices to N dimensions.

- **Scalar (0D Tensor):** A single number (e.g., the final Loss value).
- **Vector (1D Tensor):** A 1D array (e.g., a bias term).
- **Matrix (2D Tensor):** A 2D grid of numbers (e.g., a grayscale image).
- **3D Tensor:** A volume of data (e.g., a colour image $H \times W \times C$).
- **4D Tensor:** A batch of images ($B \times C \times H \times W$), used for parallel processing.

9.1.1 GPU Acceleration

Deep learning involves multiplying massive matrices (e.g., $10,000 \times 10,000$). Though each basic operation is simple, the no. of such operations is large and CPUs process these sequentially, which is slow.

- Tensors, which are designed for hardware acceleration, can be moved to the **GPU (Graphics Processing Unit)**.
- GPUs possess thousands of cores designed for parallel processing - executing hundreds of operations simultaneously.
- This parallelism can speed up matrix operations by $\approx 30\times$ compared to standard NumPy arrays on a CPU.

9.2 Calculus of Learning

Training a neural network requires adjusting millions of weights (w) to minimize the error. To do this, we need the gradient of the loss with respect to every single parameter:

$$\frac{\partial Loss}{\partial w} \tag{35}$$

For a deep network, calculating this manually via the Chain Rule is practically impossible.

9.3 Autograd: Automatic Differentiation

PyTorch solves this using the `torch.autograd` engine.

- **Dynamic Computational Graph (DAG):** As you run your Python code (Forward Pass), PyTorch builds a Directed Acyclic Graph behind the scenes.
- **Tracking History:** If a tensor is created with `requires_grad=True`, the engine records every operation performed on it.

9.3.1 The Backward Pass

Once the forward pass is complete and the Loss is calculated:

1. We call `loss.backward()`.
2. The engine traverses the graph in reverse (from Loss → Input).
3. It automatically applies the Chain Rule to compute gradients for every weight/bias created with `requires_grad=True`.
4. The results are stored in the `.grad` attribute of each tensor. (Actually, the result is added to the `.grad` attribute. So, it must be reset to zero after every epoch)

9.3.2 Controlling Gradients

- **Training Mode:** We set `requires_grad=True` for model parameters so they can be updated.
- **Inference Mode:** We use `with torch.no_grad():`. This pauses the tracking engine, reducing memory usage and speeding up computations when we don't need to train (e.g., testing the model).

9.4 The Training Workflow

9.4.1 Data Preparation

Before the loop, data is wrapped in a `Dataset` class and passed to a `DataLoader`.

- **Batching:** Stacking (`batch-size`) number of n-dimensional samples into (n + 1)-dimensional batches.
- **Shuffling:** Randomizing data to prevent learning order bias.

9.4.2 The Iterative Loop

For each batch of data:

1. **Forward Pass:** Pass inputs (x) through the model to get a prediction, $\hat{y}$.

$$\hat{y} = \text{model}(x)$$

2. **Compute Loss:** Calculate how wrong the prediction is using a loss function (e.g., MSE or Cross-Entropy).

$$L = \text{LossFn}(\hat{y}, y)$$

3. **Backward Pass:** Trigger Autograd to calculate gradients.

$$\text{loss.backward()}$$

4. **Update Weights (Optimization):** The optimizer uses the calculated gradients to adjust the weights in the direction that reduces error.

$$w_{new} = w_{old} - \eta \cdot w.grad$$